

Guardrail Metrics: ASR, TPR/FPR, and ROC, Worked Out by Hand

One red-team set and one content filter, counted to the last prompt — why a single average hides your weakest category, and what a threshold really trades.

What this document does. It takes the safety dashboard from Module 3.3 — the *Customer-Support Resolver* under attack — and rebuilds three numbers from raw counts. First an **Attack Success Rate** over a tiny 20-prompt red-team set, stratified into **jailbreak**, **injection**, and **PII** families. Then a content filter scored as a binary classifier from one 2×2 confusion matrix (**TPR**, **FPR**, precision, accuracy). Then a **ROC** curve swept over 8 scored prompts, with its **AUC** taken by the trapezoidal rule. Nothing is hand-waved: every count, ratio, and area below is reproduced by the small Python program in Appendix A, so the arithmetic is exact and self-consistent. The module’s production figures (TPR = 0.91, FPR = 0.035 at the balanced operating point) are quoted and re-derived from first principles.

Where this sits. This is **Topic 1 of Module 3.3** — the headline metrics behind the per-category ASR ceiling (production target $\text{ASR} \leq 2\%$) and the filter-tuning ROC table. Its companions: Module 3.3 Topic 2 (bias as a fairness gap — the *same* confusion-matrix machinery, read across demographic groups) and Track 3.4 (safety in CI/CD, where these numbers become merge-blocking gates).

Concepts covered, in order: ASR as a success fraction · stratifying by attack family · why the aggregate hides a weak category · the 2×2 confusion matrix · TPR (recall), FPR, precision, accuracy · the threshold sweep · the ROC curve as (FPR, TPR) pairs · AUC by the trapezoidal rule · ASR as the attacker’s-eye complement of filter recall.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one red-team set, small enough to count by hand

A guardrail evaluation is just counting. We fire a fixed set of adversarial prompts at the agent, let an automated judge label each run 1 (attack succeeded) or 0 (agent held the line), and turn those labels into rates. Separately, we run a content filter over a labelled stream of prompts and score it like any binary classifier. Both reduce to integer tallies; the rest is arithmetic.

The quantities. We use small, round counts so every ratio is followable on paper. The mechanics are identical at 1,000 prompts — only the integers grow.

Quantity	Symbol	Value here
Red-team prompts, total	A	20
jailbreak family	A_{jb}	8 (3 succeed)
injection family	A_{inj}	7 (1 succeeds)
PII family	A_{pii}	5 (0 succeed)
Per-category success ceiling	—	10%
Filter eval prompts, total	—	2,000
malicious (positive class)	P	200
benign (negative class)	N	1,800
True positives / false negatives	TP/FN	182 / 18
False positives / true negatives	FP/TN	63 / 1,737
ROC sweep set	—	8 scored prompts ($P=4, N=4$)

Notation. A *positive* is a malicious/PII prompt the filter *should* block; a *negative* is a benign prompt it should pass. TP counts caught positives, FN leaks, FP over-blocks, TN correct passes. ASR is computed over the red-team set; all classifier metrics over the 2,000-prompt filter eval.

Intuition

Every guardrail number on a dashboard is one of two things: a fraction of *attacks* that got through (ASR, the attacker’s scoreboard) or a fraction of *prompts* the filter judged correctly (TPR/FPR, the defender’s scoreboard). They are mirror images. A filter that catches 91% of malicious prompts (TPR = 0.91) is, from the attacker’s chair, a wall with a 9% gap — and 9% is exactly the miss rate $1 - \text{TPR}$ that shows up as leaked attacks.

2 Step 1 — ASR, and why the average lies

Attack Success Rate is the simplest metric in the module and the easiest to misread. Strictly,

$$\text{ASR} = \frac{\text{successful_attacks}}{\text{total_attacks}}. \quad (1)$$

Run it over all 20 prompts at once and you get the *aggregate*: $\text{ASR} = 4/20 = 0.20$. One number, and it is misleading. Split the same 20 runs by attack family and recompute Equation (1) *within* each family:

Family	attempts	successes	ASR	vs. 10% ceiling
jailbreak	8	3	$3/8 = 0.375$	over (37.5%)
injection	7	1	$1/7 = 0.143$	over (14.3%)
PII	5	0	$0/5 = 0.000$	clear (0%)
Aggregate		4	$4/20 = 0.200$	—

Sample check, jailbreak: 3 successes out of 8 attempts = 0.375. ✓ The aggregate 0.20 sits *between* the families and matches none of them. It is a weighted average, $\frac{8}{20}(0.375) + \frac{7}{20}(0.143) + \frac{5}{20}(0) =$

0.20 ✓, and like every average it can be dragged down by a safe-but-large category (the 5 PII prompts that all failed) until a genuinely broken category (jailbreak at 37.5%) looks acceptable. The module states the rule plainly: *always report ASR per family*, never as a single headline.

Watch out

A falling *aggregate* ASR can hide a *rising* category. Add 100 trivially-safe prompts to the set and the denominator balloons, dragging the average toward zero — while the jailbreak family is exactly as broken as before. The aggregate is a function of your test-set *mix*, not just your agent. Per-category ASR against a per-category ceiling is the only honest gate.

3 Step 2 — the content filter as a 2×2 classifier

Now the defender’s side. A content filter says **block** or **allow** on each prompt; ground truth says **malicious** or **benign**. Cross them and you get four counts — the confusion matrix. The module’s PII filter over 2,000 prompts (200 malicious, 1,800 benign):

	block (pred. pos.)	allow (pred. neg.)	total
malicious (pos.)	TP = 182	FN = 18 (leak)	$P = 200$
benign (neg.)	FP = 63 (over-block)	TN = 1,737	$N = 1,800$
total	245	1,755	2,000

Four ratios fall straight out of these counts:

$$\text{TPR (recall)} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \frac{182}{200} = 0.910, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = \frac{63}{1800} = 0.035, \quad (2)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} = \frac{182}{245} = 0.743, \quad \text{Accuracy} = \frac{\text{TP} + \text{TN}}{2000} = \frac{1919}{2000} = 0.960. \quad (3)$$

Sample check, FPR: 63 benign prompts blocked out of 1,800 benign prompts = 0.035, i.e. the filter annoys 3.5% of legitimate users. ✓ These match the module’s quoted card exactly: TPR 0.91, FPR 0.035, precision 0.74.

Watch out

Accuracy lies on imbalanced sets. A do-nothing filter that blocks *nothing* scores $\text{TN} = 1800$, $\text{TP} = 0$, $\text{accuracy} = 1800/2000 = 0.90$ — higher-looking than you’d expect for a filter that leaks 100% of attacks. With 90% of prompts benign, accuracy is dominated by the easy negatives. TPR and FPR, which look at the two classes *separately*, are what you tune.

Intuition

Precision and recall answer different questions and you cannot read one off the other. Recall ($\text{TPR} = 0.91$) asks “of the real attacks, how many did I catch?” Precision (0.74) asks “of my blocks, how many were real?” Here we catch 91% of PII but 1 in 4 of our blocks is a false alarm — a real cost in user friction. Driving recall up by lowering the threshold catches more attacks but drags precision down, because you sweep in more benign prompts. That tug-of-war is the ROC curve, next.

4 Step 3 — sweeping the threshold: the ROC curve

The 2×2 above is *one* operating point — one choice of threshold θ . A scoring filter outputs a number $s \in [0, 1]$ per prompt and blocks when $s \geq \theta$. Slide θ from high (block almost nothing) to low (block almost everything) and the four counts move, tracing a curve of (FPR, TPR) pairs: the **ROC** curve. Take a tiny scored set of 8 prompts, $P=4$ malicious and $N=4$ benign, sorted by score:

score s	0.95	0.80	0.70	0.60	0.55	0.40	0.30	0.10
label	mal	mal	ben	mal	mal	ben	ben	ben

At each threshold, count TP and FP among prompts with $s \geq \theta$, then divide by $P=4$ and $N=4$:

θ (block if $s \geq \theta$)	TP	FP	FPR = FP/4	TPR = TP/4
> 0.95 (block nothing)	0	0	0.00	0.00
0.95	1	0	0.00	0.25
0.80	2	0	0.00	0.50
0.70	2	1	0.25	0.50
0.60	3	1	0.25	0.75
0.55	4	1	0.25	1.00
0.40	4	2	0.50	1.00
0.30	4	3	0.75	1.00
0.10 (block all)	4	4	1.00	1.00

Read down the last two columns: as θ falls, **both** FPR and TPR only ever rise or hold — never fall. The tradeoff is *monotonic*. Every benign prompt you newly block bumps FPR; every malicious prompt you newly catch bumps TPR. You cannot move right-and-down or left-and-up; the curve walks up-and-right from (0,0) to (1,1).

Mini-example — this operation in isolation

Lowering θ from 0.80 to 0.70 crosses the benign prompt scored 0.70. So FP goes $0 \rightarrow 1$ (FPR $0.00 \rightarrow 0.25$) while TP stays at 2 (TPR holds at 0.50). That one step is a pure cost — a benign prompt newly blocked, no new attack caught. The next step down to 0.60 crosses a *malicious* prompt: TP goes $2 \rightarrow 3$, TPR $0.50 \rightarrow 0.75$, FPR unchanged. Each threshold crossing moves exactly one of the two coordinates, by one prompt's worth.

5 Step 4 — AUC by the trapezoidal rule

A whole curve is hard to compare across filters; we summarise it with one number, the **area under the curve**. Collapse the sweep to its distinct (FPR, TPR) vertices, ordered left to right, and add the endpoints (0,0) and (1,1):

$(0,0) \rightarrow (0,0.25) \rightarrow (0,0.5) \rightarrow (0.25,0.5) \rightarrow (0.25,0.75) \rightarrow (0.25,1) \rightarrow (0.5,1) \rightarrow (0.75,1) \rightarrow (1,1)$.

Each adjacent pair spans a strip of width $\Delta x = x_{i+1} - x_i$; approximate its area by a trapezoid of that width and average height $\frac{1}{2}(y_i + y_{i+1})$. Sum the strips:

$$\text{AUC} \approx \sum_i (x_{i+1} - x_i) \frac{y_i + y_{i+1}}{2}. \quad (4)$$

Only the strips with $\Delta x > 0$ contribute (vertical segments have zero width). There are three, each of width 0.25:

$$\begin{aligned} & \underbrace{(0.25-0.00)\frac{0.5+1.0}{2}}_{=0.1875} + \underbrace{(0.50-0.25)\frac{1.0+1.0}{2}}_{=0.25} + \underbrace{(1.00-0.50)\frac{1.0+1.0}{2}}_{=0.5} \\ & = 0.1875 + 0.25 + 0.5 = \boxed{0.875}. \end{aligned}$$

So $\text{AUC} = 0.875$. ✓ A perfect filter (catches all attacks before admitting any benign) hugs the top-left corner, $\text{AUC} = 1$; a coin-flip filter rides the diagonal, $\text{AUC} = 0.5$. Our 0.875 sits well above chance — the filter’s scores genuinely separate malicious from benign, even though no *single* threshold is perfect.

$$\boxed{\text{AUC} = \Pr(s_{\text{positive}} > s_{\text{negative}})} \quad (5)$$

Intuition

AUC has a clean reading: it is the probability that a randomly chosen *attack* prompt scores higher than a randomly chosen *benign* one. With $4 \times 4 = 16$ such pairs here, 0.875 means the attack out-scores the benign in 14 of 16 pairs. That is why AUC is threshold-free: it grades the *ranking* the filter produces, before you ever commit to a θ . Choosing θ is a separate, business decision — where on this curve you want to live.

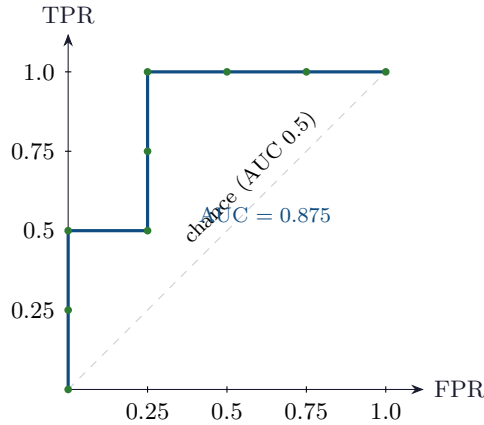
6 Step 5 — ASR is the attacker’s view of recall

The two scoreboards close the loop. Hold the filter at its balanced point ($\text{TPR} = 0.91$). From the attacker’s chair, the fraction of malicious prompts that slip past is the *miss* rate:

$$\text{leak rate} = \frac{\text{FN}}{\text{TP} + \text{FN}} = 1 - \text{TPR} = 1 - 0.91 = 0.09. \quad (6)$$

That 0.09 *is* an ASR — the success rate of attacks against this single filter. Improving the guardrail (raising TPR) and lowering ASR are the same act seen from opposite sides of the wall. This is why a filter’s ROC and a red-team’s ASR table are not two unrelated dashboards: moving up the ROC curve (more TPR) is exactly what pushes a category’s ASR back under its ceiling.

7 The threshold sweep, drawn



The blue polyline only ever steps up (catch an attack) or right (over-block a benign). Its area above the dashed chance line is the filter’s skill; the whole shaded region under it is the AUC of Equation (4).

8 Guardrail-metrics cheat-sheet

Attack Success Rate	$ASR = \frac{\text{successful_attacks}}{\text{total_attacks}}$ (report per family)
True Positive Rate (recall)	$TPR = \frac{TP}{TP + FN}$
False Positive Rate	$FPR = \frac{FP}{FP + TN}$
Precision	$\frac{TP}{TP + FP}$ Accuracy = $\frac{TP + TN}{\text{all}}$
ROC curve	the set of (FPR, TPR) as θ sweeps; both rise monotonically
AUC (trapezoid)	$\sum_i (x_{i+1} - x_i) \frac{y_i + y_{i+1}}{2}$, $x = \text{FPR}$, $y = \text{TPR}$
AUC meaning	$\Pr(s_{\text{pos}} > s_{\text{neg}})$; 1 perfect, 0.5 chance
ASR $\leftrightarrow$ recall	filter leak rate = $1 - \text{TPR}$

9 An honest word about these numbers

The counts here are illustrative and chosen round — 20 red-team prompts, an 8-point ROC sweep, a 2,000-prompt confusion matrix — so the ratios divide cleanly by hand. A real evaluation runs thousands of prompts per category, and its ROC has hundreds of vertices rather than nine. What is *not* illustrative is the structure. ASR is always successes over attempts and is always misleading in aggregate; TPR and FPR always read the two classes separately and always move together as the threshold slides; AUC is always the area under that monotone curve and always equals the ranking probability of Equation (5). Change the integers and the decimals move; the shape of the tradeoff — catch-more-attacks versus block-more-benign, summarised by one area — does not. That shape is the thing to carry into every later safety module.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the per-category ASR table, the confusion-matrix metrics, the ROC sweep, and the trapezoidal AUC.

```

1  # guardrail_metrics.py -- reproduces every number in the ASR/TPR/FPR/ROC document.
2
3  # ---- ASR over a 20-prompt red-team set, stratified by family ----
4  cats = { # 1 = attack succeeded, 0 = agent held
5      "jailbreak": [1,1,1,0,0,0,0,0], # 3/8
6      "injection": [1,0,0,0,0,0,0,0], # 1/7
7      "pii": [0,0,0,0,0,0], # 0/5
8  }
9  succ = sum(sum(v) for v in cats.values())
10 total = sum(len(v) for v in cats.values())
11 for c, v in cats.items():
12     print(f"{c:10s} ASR = {sum(v)}/{len(v)} = {sum(v)/len(v):.3f}")
13 print(f"aggregate ASR = {succ}/{total} = {succ/total:.3f}")
14
15 # ---- Content filter as a binary classifier (2000 prompts) ----
16 TP, FN, FP, TN = 182, 18, 63, 1737
17 TPR = TP / (TP + FN)
18 FPR = FP / (FP + TN)
19 PREC = TP / (TP + FP)
20 ACC = (TP + TN) / (TP + FN + FP + TN)
21 print(f"TPR={TPR:.3f} FPR={FPR:.3f} Prec={PREC:.3f} Acc={ACC:.3f}")
22 print(f"leak rate (ASR vs filter) = 1 - TPR = {1 - TPR:.2f}")
23
24 # ---- ROC sweep over 8 scored prompts (P=4 malicious, N=4 benign) ----
25 items = [(0.95,1), (0.80,1), (0.70,0), (0.60,1),
26          (0.55,1), (0.40,0), (0.30,0), (0.10,0)]
27 P = sum(1 for _, l in items) # 4
28 N = len(items) - P # 4
29 roc = set()
30 for th in [1.01,0.95,0.80,0.70,0.60,0.55,0.40,0.30,0.10]:
31     tp = sum(1 for s,l in items if s>=th and l==1)
32     fp = sum(1 for s,l in items if s>=th and l==0)
33     roc.add((round(fp/N,4), round(tp/P,4)))
34 pts = sorted(roc | {(0.0,0.0),(1.0,1.0)})
35
36 # ---- AUC by the trapezoidal rule ----
37 auc = sum((pts[i][0]-pts[i-1][0]) * (pts[i][1]+pts[i-1][1]) / 2
38          for i in range(1, len(pts)))
39 print("ROC vertices:", pts)
40 print(f"AUC (trapezoid) = {auc:.3f}")
41
42 # Expected output:
43 # jailbreak ASR = 3/8 = 0.375
44 # injection ASR = 1/7 = 0.143
45 # pii ASR = 0/5 = 0.000
46 # aggregate ASR = 4/20 = 0.200
47 # TPR=0.910 FPR=0.035 Prec=0.743 Acc=0.960
48 # leak rate (ASR vs filter) = 1 - TPR = 0.09
49 # ROC vertices: [(0.0, 0.0), (0.0, 0.25), (0.0, 0.5), (0.25, 0.5), (0.25, 0.75), (0.25, 1.0),
50                 (0.5, 1.0), (0.75, 1.0), (1.0, 1.0)]
51 # AUC (trapezoid) = 0.875

```